

Java Backend Architecture

Interview Series

Chapter 10.3

Index Mappings & Analyzers

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

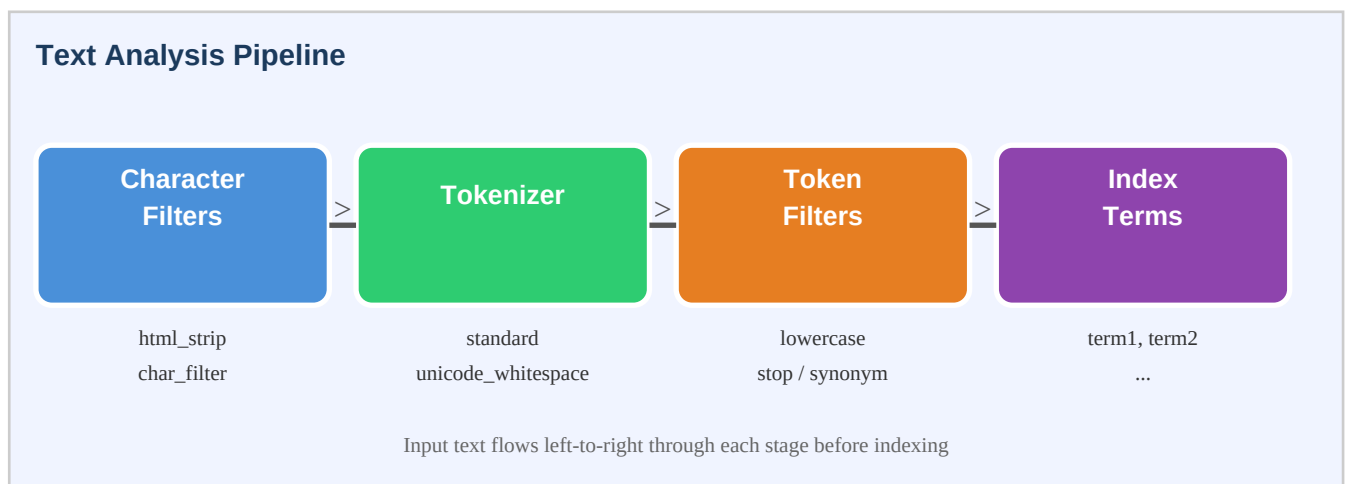
Chapter 10.3 – Index Mappings & Analyzers

Introduction

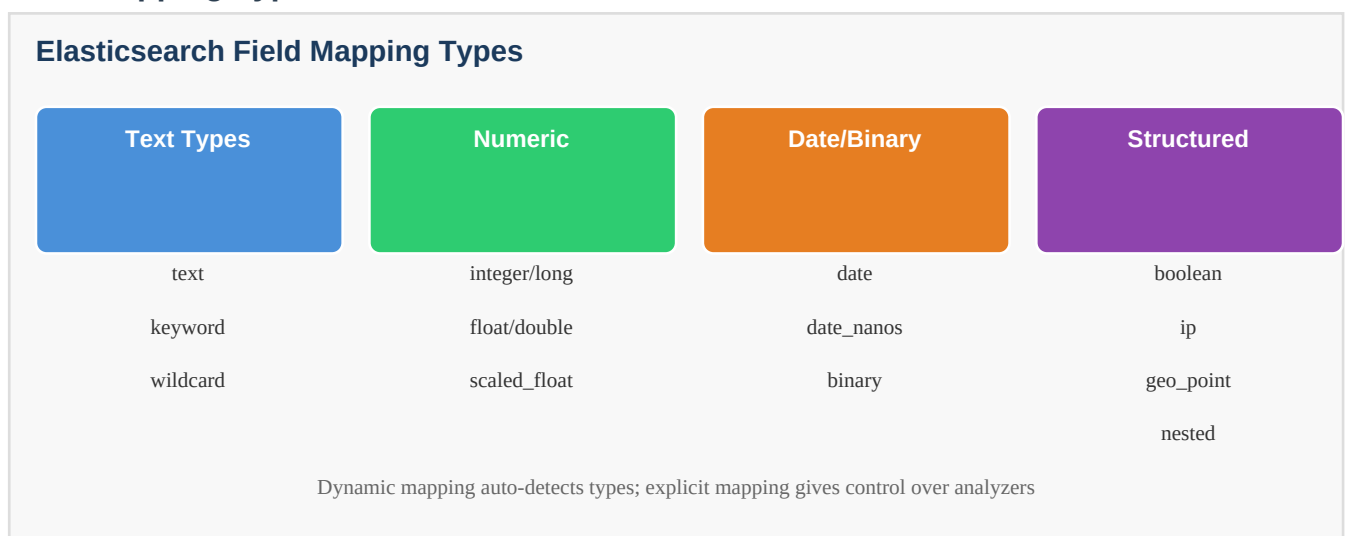
Index mappings define how documents and their fields are stored and indexed in Elasticsearch. An analyzer determines how text is broken into searchable tokens. Together, mappings and analyzers form the foundation of a well-tuned search application. Choosing the wrong field type or analyzer at index time is difficult to reverse — you must reindex — making upfront design critical.

Elasticsearch supports two mapping strategies: dynamic (auto-detect) and explicit (manual). Production systems should always use explicit mappings to avoid accidental type coercions, mapping explosions, and unwanted keyword/text sub-fields that balloon storage.

Text Analysis Pipeline



Field Mapping Types



Dynamic vs Explicit Mapping

Dynamic vs Explicit Mapping

Dynamic Mapping

- + Auto-detects field types
- + Convenient for prototyping
- + Risk: string as text+keyword
- + Set dynamic: strict to disable

Explicit Mapping

- + Full control over types
- + Specify analyzers per field
- + Prevent mapping explosions
- + Required in production

VS

Tip: Start dynamic, graduate to explicit mapping before production deployment

Key Definitions

Mapping	Schema definition of an index — field names, types, and index options
Dynamic Mapping	ES auto-infers types from JSON values; convenient but risky in production
Explicit Mapping	Manually defined field types, analyzers, and index settings
Analyzer	A combination of character filters, a tokenizer, and token filters
Tokenizer	Splits text into tokens (e.g., standard, whitespace, ngram)
Token Filter	Transforms tokens after tokenization (lowercase, stop, synonym, stemmer)
Character Filter	Transforms raw text before tokenization (html_strip, pattern_replace)
text	Full-text field, analyzed by an analyzer; not usable for aggregations
keyword	Exact-match string field; not analyzed; used for sorting/aggregations/filters
normalizer	Like an analyzer but for keyword fields (no tokenization, just filters)
multi-field	A single field indexed in multiple ways (e.g., text + keyword sub-field)
dynamic: strict	Rejects documents that contain unmapped fields; prevents mapping drift
index_options	Controls what is stored in the inverted index (docs, freqs, positions, offsets)
doc_values	Column-oriented on-disk structure for sorting/aggregations on keyword/numeric
fielddata	In-memory structure enabling aggregations on analyzed text fields (expensive)

Built-in Analyzers

Analyzer	Tokenizer	Filters Applied	Best For
standard	Standard (Unicode)	lowercase, stop words	General English text
simple	Lowercase	none	Simple tokenization
whitespace	Whitespace	none	Code/logs/IDs
keyword	Single token	none	Exact-match passthrough

english	Standard	lowercase, possessive, stop, pluralization	English language search
fingerprint	Standard	lowercase, stop, fingerprint	Deduplication

Code Examples

1. Creating an index with explicit mappings

```
// PUT /products { "settings": { "number_of_shards": 3, "number_of_replicas": 1, "analysis": {
"analyzer": { "my_english": { "type": "english" }, "autocomplete_analyzer": { "type": "custom",
"tokenizer": "standard", "filter": ["lowercase", "autocomplete_filter"] } } }, "filter": {
"autocomplete_filter": { "type": "edge_ngram", "min_gram": 2, "max_gram": 20 } } } },
"mappings": { "dynamic": "strict", "properties": { "id": { "type": "keyword" }, "name": {
"type": "text", "analyzer": "my_english", "fields": { "keyword": { "type": "keyword",
"ignore_above": 256 }, "suggest": { "type": "text", "analyzer": "autocomplete_analyzer" } } },
"description": { "type": "text", "analyzer": "my_english" }, "price": { "type": "scaled_float",
"scaling_factor": 100 }, "category": { "type": "keyword" }, "tags": { "type": "keyword" },
"created_at": { "type": "date", "format": "strict_date_optional_time" }, "in_stock": { "type":
"boolean" }, "location": { "type": "geo_point" } } } }
```

2. Java: Creating index with RestHighLevelClient

```
// Using Elasticsearch Java High Level REST Client import
org.elasticsearch.client.indices.CreateIndexRequest; import
org.elasticsearch.client.indices.CreateIndexResponse; import
org.elasticsearch.common.settings.Settings; import org.elasticsearch.xcontent.XContentBuilder;
import static org.elasticsearch.xcontent.XContentFactory.jsonBuilder; public class
IndexCreator { private final RestHighLevelClient client; public void createProductIndex()
throws IOException { CreateIndexRequest request = new CreateIndexRequest("products"); //
Settings request.settings(Settings.builder().put("index.number_of_shards", 3)
.put("index.number_of_replicas", 1).build()); // Mappings via builder XContentBuilder
mappings = jsonBuilder().startObject().field("dynamic", "strict").startObject("properties")
.startObject("id").field("type", "keyword").endObject().startObject("name").field("type",
"text").field("analyzer", "english").startObject("fields").startObject("keyword")
.field("type", "keyword").field("ignore_above", 256).endObject().endObject().startObject("price")
.field("type", "scaled_float").field("scaling_factor", 100)
.endObject().startObject("category").field("type", "keyword").endObject()
.startObject("created_at").field("type", "date").field("format",
"strict_date_optional_time").endObject().endObject().endObject();
request.mapping(mappings); CreateIndexResponse response = client.indices().create(request,
RequestOptions.DEFAULT); if (!response.isAcknowledged()) { throw new RuntimeException("Index
creation not acknowledged"); } }
```

3. Java: Spring Data ES – @Document & @Field annotations

```
import org.springframework.data.annotation.Id; import
org.springframework.data.elasticsearch.annotations.*; @Document(indexName = "products",
createIndex = true) @Setting(settingPath = "/es-settings.json") // custom analyzer public class
ProductDocument { @Id private String id; @Field(type = FieldType.Text, analyzer = "english",
searchAnalyzer = "english") @MultiField(mainField = @Field(type = FieldType.Text, analyzer =
"english"), otherFields = { @InnerField(suffix = "keyword", type = FieldType.Keyword),
@InnerField(suffix = "suggest", type = FieldType.Text, analyzer = "autocomplete_analyzer") })
private String name; @Field(type = FieldType.ScaledFloat, scalingFactor = 100) private
BigDecimal price; @Field(type = FieldType.Keyword) private String category; @Field(type =
```

```
FieldType.Date, format = DateFormat.strict_date_optional_time) private Instant createdAt;
@Field(type = FieldType.Boolean) private boolean inStock; // getters/setters omitted }
```

4. Testing the analyzer with `_analyze` API

```
// POST /products/_analyze { "analyzer": "my_english", "text": "The quick brown foxes are
jumping over lazy dogs" } // Response tokens: [quick, brown, fox, jump, over, lazi, dog] //
Note: stemming applied (foxes->fox, jumping->jump, lazy->lazi) // Test a custom analyzer during
design: // POST /_analyze { "tokenizer": "standard", "filter": ["lowercase", "stop",
"porter_stem"], "text": "Running faster than the fastest runners" } // Tokens: [run, faster,
fastest, runner] // Java: call _analyze for debugging Request analyzeRequest = new
Request("POST", "/products/_analyze"); analyzeRequest.setJsonEntity("{\"
{ \"analyzer\": \"my_english\", \"text\": \"Running products quickly\" } \"\""); Response response =
client.getLowLevelClient().performRequest(analyzeRequest); // parse response to verify tokens
```

5. Reindex after mapping change

```
// Mappings are immutable for existing fields – you must reindex. // Step 1: Create new index
with updated mapping // PUT /products_v2 { ... new mapping ... } // Step 2: Reindex via API //
POST /_reindex { "source": { "index": "products_v1" }, "dest": { "index": "products_v2" } } //
Step 3: Update alias atomically // POST /_aliases { "actions": [ { "remove": { "index":
"products_v1", "alias": "products" } }, { "add": { "index": "products_v2", "alias": "products"
} } ] } // Java: Reindex programmatically ReindexRequest reindex = new ReindexRequest();
reindex.setSourceIndices("products_v1"); reindex.setDestIndex("products_v2");
reindex.setRefresh(true); BulkByScrollResponse response = client.reindex(reindex,
RequestOptions.DEFAULT); System.out.println("Reindexed: " + response.getTotal());
```

6. Synonym filter configuration

```
// In index settings: "analysis": { "filter": { "synonym_filter": { "type": "synonym",
"synonyms": [ "laptop, notebook, computer", "cell phone, mobile phone, smartphone", "tv =>
television, screen" ] } }, "analyzer": { "synonym_analyzer": { "tokenizer": "standard",
"filter": ["lowercase", "synonym_filter"] } } } // For large synonym files use synonym_path:
"synonym_filter": { "type": "synonym", "synonyms_path": "analysis/synonyms.txt" // File must
exist in ES config/analysis/ directory }
```

Interview Questions & Answers

Q1. What is the difference between a `text` and a `keyword` field?

`text` fields are analyzed (tokenized, lowercased, stemmed) and support full-text search. `keyword` fields are stored as-is (not analyzed) and support exact-match queries, sorting, and aggregations. Use `text` for human-readable prose, `keyword` for IDs, categories, email addresses, and any field you filter/sort/aggregate on.

Q2. What are multi-fields and when would you use them?

Multi-fields allow the same source value to be indexed in multiple ways. For example, a `name` field can be indexed as `text` (for full-text search), `name.keyword` (for exact match and aggregation), and `name.suggest` (for autocomplete). You define them under `fields` in the mapping. This avoids duplicating data in the source document.

Q3. Why are mappings in Elasticsearch largely immutable?

Because an existing field's indexed data is already stored in the inverted index with its original analysis. Changing a field type would invalidate existing data — ES cannot retroactively re-analyze stored documents. To change a field type, create a new index with the updated mapping and reindex the data.

Q4. Explain the three components of an analyzer.

1) Character filters pre-process raw text (e.g., ``html_strip`` removes HTML tags, ``pattern_replace`` applies regex substitutions). 2) A tokenizer splits text into tokens (e.g., ``standard`` splits on whitespace/punctuation). 3) Token filters transform tokens (e.g., ``lowercase``, ``stop``, ``stemmer``, ``synonym``). All three are applied sequentially during indexing and query analysis.

Q5. What is ``dynamic: strict`` and why use it in production?

``dynamic: strict`` causes ES to reject any document that contains fields not defined in the explicit mapping, throwing a ``strict_dynamic_mapping_exception``. This prevents: (1) accidental mapping explosions from deeply nested JSON, (2) incorrect type inference (e.g., a numeric string mapped as ``long`` instead of ``keyword``), and (3) unpredictable index growth. In production, always define your schema explicitly and set `dynamic: strict``.

Q6. What is a mapping explosion and how do you prevent it?

A mapping explosion occurs when thousands of unique field names are dynamically added (e.g., from JSON keys that include user IDs). ES stores the full mapping in cluster state (memory), so thousands of field mappings can cause OOM errors on master nodes. Prevention: use ``dynamic: strict`` or ``dynamic: false``, use ``flattened`` field type for arbitrary key-value pairs, or model dynamic keys as nested objects with a known structure.

Q7. Difference between ``doc_values`` and ``fielddata``?

``doc_values`` is an on-disk column-oriented data structure built at index time for keyword/numeric/date/geo fields — used for sorting and aggregations. It is efficient because it doesn't consume heap. ``fielddata`` is a lazy in-heap structure built on-demand for ``text`` fields to support aggregations. It is memory-intensive and can cause OOM errors. Prefer ``keyword`` fields over enabling `fielddata` on text fields.

Q8. How do you test whether an analyzer tokenizes text correctly?

Use the ``_analyze`` API: ``POST /my-index/_analyze`` with ``analyzer`` and ``text`` fields. You can also test a not-yet-created analyzer inline by specifying ``tokenizer`` and ``filter`` arrays. The response lists each token with its type, position, and character offsets. Always test custom analyzers with representative sample text before going to production.

Q9. What is a normalizer and how does it differ from an analyzer?

A normalizer is like an analyzer but for ``keyword`` fields. It applies character and token filters but no tokenizer — the entire value is treated as a single token. This lets you normalize keyword fields for case-insensitive matching (lowercase normalizer) or accent-folding without affecting the original stored value.

Q10. Explain the ``english`` analyzer and its stemming behavior.

The ``english`` built-in analyzer uses: possessive stemmer (removes 's), standard tokenizer, lowercase filter, English stop words filter, and Porter stemmer. The Porter stemmer reduces words to their root form (e.g., ``running`` -> ``run``, ``better`` -> ``better``, ``foxes`` -> ``fox``). This improves recall at the cost of precision. Use ``english`` for general English content; for multi-language use ICU analysis plugin.

Q11. What is the ``ignore_above`` setting on a keyword field?

``ignore_above: N`` means strings longer than N characters are NOT indexed or stored in the keyword field (the original ``_source`` is unaffected). This prevents extremely long strings from entering the keyword index (which has no meaningful exact-match use for very long strings anyway). Common value is 256. Useful on

`*.keyword`` sub-fields of text fields where the full text isn't needed for aggregations.

Q12. How do you add a new field to an existing index without reindexing?

You can add new fields to an existing mapping using `PUT /my-index/_mapping`` with the new field definition. ES supports adding fields dynamically to existing mappings. However, existing documents won't have values for the new field until they're updated/reindexed. You CANNOT change the type of an existing field — that requires a full reindex.

Q13. What is the difference between ``index`` and ``store`` in field mapping?

``index: false`` means the field is NOT added to the inverted index — it can't be searched but is still in ``_source``. ``store: true`` means the field value is stored separately from ``_source`` and can be retrieved individually (faster than parsing the full ``_source``). By default, ``index: true`` and ``store: false`` — ES retrieves values from ``_source``.

Q14. When would you use the ``flattened`` field type?

The ``flattened`` type maps an entire JSON object as a single field. All leaf values are indexed as keywords under one field name. It prevents mapping explosions from objects with arbitrary keys (e.g., metadata, custom attributes, log labels). Limitation: you can only do exact-match queries; no full-text or range queries on sub-keys.

Q15. Explain ``copy_to`` in field mappings.

``copy_to`` allows values from multiple fields to be copied into a single target field, enabling multi-field search on that combined field. The target field is searchable but not stored in ``_source``. Example: copy ``first_name`` and ``last_name`` into a ``full_name`` field to allow ``full_name:John Doe`` queries without needing a runtime script.

Q16. What are runtime fields and when should you use them?

Runtime fields are computed at query time from existing fields using Painless scripts. They don't exist in the index — they're calculated on the fly. Use them when you need to: (1) add a derived field without reindexing, (2) test a new field before committing to a mapping change, (3) override indexed values for specific queries. Downside: they are slower than indexed fields since they're computed per query.

Q17. How do synonyms work in ES analysis and what are common pitfalls?

A ``synonym`` token filter expands or contracts tokens using a synonym file or inline list. Expansion: `'laptop' -> 'laptop, notebook, computer'`. Contraction: `'laptop, notebook => laptop'`. Common pitfalls: (1) synonyms at index time vs. query time — prefer query-time synonyms to avoid reindexing on synonym changes; (2) position increments with multi-word synonyms can cause phrase query mismatches; (3) synonym files must be identical across all nodes.

Q18. What is ``index_options`` and how does it affect query performance?

``index_options`` controls what information is stored in the inverted index: ``docs`` (doc IDs only), ``freqs`` (+ term frequency), ``positions`` (+ token positions for phrase queries), ``offsets`` (+ character offsets for highlighting). More data = better query capability but higher disk use. Default for ``text`` is ``positions``; reducing to ``freqs`` saves space if you don't need phrase queries.

Q19. What is the ``scaled_float`` field type and when is it preferred over ``float``?

``scaled_float`` stores floating-point numbers as long integers scaled by a fixed factor (e.g., ``scaling_factor: 100`` stores 19.99 as 1999). This is more storage-efficient than IEEE 754 floats for values with known precision (prices, percentages). It uses ``doc_values`` as longs, making aggregations and sorting faster. Prefer ``scaled_float`` for currency and percentage fields where precision is fixed.

Q20. Describe the reindex workflow for a breaking mapping change.

1) Create a new index (`products_v2`) with the corrected mapping. 2) Use the `_reindex` API to copy documents from `products_v1` to `products_v2` — optionally with a script to transform documents. 3) Validate the new index (doc count, field data, sample queries). 4) Atomically swap aliases: remove alias from v1, add to v2 in a single `_aliases` call. 5) Delete v1 after confirming v2 is healthy. Always use aliases in application code.

Q21. How do you handle multi-language text analysis in Elasticsearch?

Options: (1) Use language-specific built-in analyzers (english, french, german) per index or per field. (2) Use the ICU Analysis plugin for Unicode-aware tokenization and normalization across scripts. (3) Store each language in a separate field with its own analyzer. (4) Use the `language` field to route queries to the correct analyzer via scripted queries. Multi-language in a single field is challenging — separate fields or separate indices per language are most maintainable.

Q22. What happens to a document if `dynamic: strict` is set and it has an unmapped field?

ES throws a `strict_dynamic_mapping_exception` and rejects the entire document — it is NOT partially indexed. The error includes the offending field name. To handle this in a Java indexing pipeline, catch the exception, log the rejected document for inspection, and either strip unknown fields before re-indexing or update the mapping first.

Q23. Explain `nested` vs `object` field types for arrays of objects.

`object` type flattens array-of-object fields — all sub-field values across array elements are merged, so cross-element queries can give false positives. `nested` type keeps each element as a separate hidden document, enabling accurate per-element queries with `nested` query and `nested` aggregations. Nested fields are more expensive (extra documents in the index) but essential for accurate modeling of one-to-many relationships.

Q24. What is the `eager_global_ordinals` setting on keyword fields?

Global ordinals are an in-memory structure that maps each unique keyword value to an ordinal integer, used for aggregations. By default they are built lazily on the first aggregation after a segment refresh. `eager_global_ordinals: true` rebuilds them at refresh time, ensuring the first aggregation request after new data is just as fast as subsequent ones. Enable on high-cardinality keyword fields that are frequently aggregated.

Q25. How do you optimize an index for write-heavy workloads?

1) Increase `refresh_interval` (e.g., `30s`) to reduce segment creation overhead. 2) Disable `_source` compression or reduce stored fields. 3) Set `index.translog.durability: async` (risks small data loss on crash). 4) Use bulk indexing API with optimal batch sizes (5-15 MB). 5) Disable `doc_values` on fields not used for sorting/aggregation. 6) Set `index: false` on fields not used for search. 7) Use `index.codec: best_compression` only when disk is the bottleneck.

Q26. What are the best practices for mapping field naming conventions?

1) Use `snake_case` for field names (consistent with most ES defaults). 2) Avoid dots in field names — they can conflict with object hierarchy. 3) Don't use `_` prefix (reserved by ES for metadata fields). 4) Prefix all timestamp fields consistently (`created_at`, `updated_at`). 5) Use explicit `boolean` fields (not string `'true'/'false'`). 6) Document your mapping in code alongside the `@Document` annotation. 7) Version your index name (`products_v1`, `products_v2`) and use aliases.

Q27. Describe the `search_analyzer` setting and when it differs from `analyzer`.

``analyzer`` is used at index time to tokenize and normalize text before storage. ``search_analyzer`` is used at query time to tokenize the search query. By default, both use the same analyzer. You might differ them when: (1) using `edge_ngram` for autocomplete — index with ``edge_ngram_analyzer``, search with ``standard`` to avoid matching on partial ngrams from the query; (2) adding synonyms at query time only to avoid reindexing when synonym lists change.

Q28. What is a ``dense_vector`` field and when is it used?

``dense_vector`` stores arrays of float values (embeddings) for k-nearest neighbor (kNN) search. Used in semantic/vector search scenarios where documents and queries are represented as embeddings from ML models (BERT, OpenAI, etc.). Requires ``dims`` to match the embedding dimension. Query with ``knn`` query or ``script_score`` for exact/approximate nearest neighbor search. Available since ES 7.3; approximate kNN using HNSW since 8.0.

Q29. How do index templates work and why are they important?

Index templates define settings and mappings that are automatically applied to new indices whose names match a pattern. Component templates are reusable building blocks; index templates compose them. Important for time-series indices (e.g., `logs-*`, `metrics-*`) created by Logstash or Beats, ensuring consistent mappings/settings without manual index creation. In Java, define templates via ``PUT /_index_template/my-template``.

Q30. What is the ``_meta`` field in mappings and how do you use it?

``_meta`` is a free-form object stored with the mapping for user-defined metadata — ES ignores it during indexing/querying. Common uses: versioning the mapping schema (e.g., ``{`schema_version`: `2.1`}``), recording which service owns the index, storing the last-updated timestamp, or linking to documentation. Retrieve it via ``GET /my-index/_mapping``.

Q31. Explain what ``enabled: false`` does in a mapping.

``enabled: false`` on an object field tells ES to store the JSON object in ``_source`` but not parse or index it at all — no field mapping entries, no inverted index entries. The field cannot be searched or aggregated. Use it for large blobs of metadata that you need to retrieve but never search (e.g., raw API response, audit payload). Saves significant indexing overhead for complex nested structures.

Q32. What are the implications of setting ``store: true`` on a field?

With ``store: true``, the field value is stored separately from ``_source`` in a column-oriented store. Benefits: (1) you can retrieve just that field without fetching and parsing the full ``_source`` JSON, which matters for very large ``_source`` documents; (2) it can be used for highlighting even if ``_source`` is disabled. Downside: additional disk space. Most use cases don't need it — fetching from ``_source`` is efficient enough. Useful for large ``_source`` with many fields when only a few are frequently retrieved.

Q33. How do you disable ``_source`` and what are the trade-offs?

Set ``_source: {enabled: false}`` in the mapping. This stops storing the original JSON, saving significant disk space. Trade-offs: (1) you cannot use `reindex` API without a remote source; (2) `update/partial update` API doesn't work; (3) highlighting requires ``store: true`` on individual fields; (4) `script updates` using ``_source`` don't work. Only disable ``_source`` in space-constrained scenarios with immutable documents.

Q34. How would you design a mapping for a product catalog with multi-language support?

Use per-language fields with appropriate analyzers: ``name_en`` (analyzer: `english`), ``name_fr`` (analyzer: `french`), ``name_de`` (analyzer: `german`). Each with a ``keyword`` sub-field for exact match. Add a ``locale`` keyword field. At query time, select fields based on the user's locale. Use index templates to enforce consistent mapping across language-specific indices. For search across languages, use ``multi_match`` with ``fields: [name_en, name_fr, name_de]``.

Q35. What happens if two fields have the same name across different nested objects in ES?

In ES, field names within a mapping must be unique at each nesting level, but you can have fields with the same name in different object paths (e.g., ``address.city`` and ``shipping.city``). They are treated as completely separate fields. However, if you use ``copy_to`` to combine them, specify both source fields. Avoid ambiguity by using full dotted path in queries.